BrewFS: Delta-Based Multi-Agent Workspaces with Packed Metadata and Variable-Granularity Object Storage

Internal research draft
22 October 2026

Abstract

Parallel coding agents can start from a common repository and dependency environment while requiring independent file-system views. Reusing that baseline raises three connected questions: how to represent private changes, how to access shared metadata without repeated per-file database lookups, and how to map partial object access to physical storage I/O. We present the design of BrewFS, which combines delta-based workspaces, packed immutable metadata, and an interface for per-object storage layouts. A private writable upper records namespace and data-mapping changes over a pinned snapshot. The shared lower places directory entries, common attributes, and data locators in paged, directory-oriented structures on object storage. Once an upload's final length is known, a layout hint allows the backend to select and persist an encoding block and a compatible verification/read quantum. We describe inspected prototype components and identify the remaining integration gates. We also specify an evaluation that separates layout effects from caching and request coalescing, accounts for construction and recovery costs, and tests complete agent tasks. The present draft reports no new performance results.

Status. This is an author working document: design and source audit are distinguished from planned experiments. Integration, correctness, and performance claims require the evidence described below.

1 Introduction

Coding agents interact with repositories by navigating directories, reading files, editing code, and invoking builds and tests. Systems such as SWE-agent and OpenHands make these interactions part of a repeated tool-use loop [21, 24]. Running several such tasks from the same repository version creates an opportunity to share their initial state. Each task nevertheless needs an independent view: an edit, deletion, or generated file in one workspace must not silently change another task's input. The opportunity is particularly relevant when workspaces are provisioned on different clients and their durable state resides in object storage.

Sharing file contents addresses only part of the cost. A workspace also needs a namespace, file attributes, and mappings from file offsets to stored data. Materializing these records for every task can repeat work even when most of the baseline is unchanged. Keeping a shared baseline in a conventional mutable metadata service can instead make its read path depend on per-file queries. Moving metadata to objects introduces another trade-off: fetching an index or a group of entries saves some lookups but may transfer unused metadata or file data. Finally, a short request at the file-system interface need not become a short physical read in an erasure-coded object store.

These issues do not make copy-on-write workspaces or object-resident metadata new abstractions. OverlayFS already provides layered namespace semantics [12]; BranchFS applies copy-on-write branches to agentic exploration [20]. DeltaFS places private namespace changes and immutable logs over shared storage [26]. Immutable distribution systems also provide indexed, demand-loaded baselines [2, 14]. The research question for BrewFS is therefore narrower: *which representation and access boundaries make shared agent workspaces inexpensive when their metadata and data must be fetched from remote objects?* Answering it requires comparisons with these approaches, rather than treating full directory copying as the only alternative.

Our design separates three decisions that become possible at different times. When a workspace is created, it can pin a shared immutable lower and allocate only its private state. When a read-only subtree is published, its directory entries, common attributes, and data locators can be organized together for subsequent access. When an object is finalized for upload, its actual wire length is known and can inform the backend's layout. Keeping these boundaries explicit also identifies the state that still requires coordination: mutable upper records, snapshot publication, and reader ownership remain separate from immutable lower lookup.

BrewFS implements components of this design in three

parts. First, *delta-based workspaces* resolve a pinned baseline and private current state into an independent view. The private state includes deletion markers and data mappings; it is not a byte-delta compression format. Second, *packed metadata* represents selected read-only subtrees as immutable snapshots indexed by directory-name ranges. The reader can resolve supported lower namespace, attribute, and data-location queries from those objects. Third, *variable-granularity object storage* uses a BrewFS–RustFS interface to request an encoding block for a finalized object. RustFS validates the request and persists the effective encoding and verification geometry for subsequent reads and repair.

The combination is a testable hypothesis, not an assumed multiplicative speedup. A sparse private upper can preserve reuse of packed lower pages; packing can in turn create objects whose partial access benefits from finer physical verification. Conversely, frequent subtree rebuilding can defeat amortization, an upper read may still fetch a complete object for integrity, and whole-object caching can hide or reverse a backend layout benefit. The evaluation must expose these cases.

This working draft makes three contributions at the level of design and research preparation. It defines the state and lookup contracts that connect private workspaces to shared packed snapshots; it describes a per-object layout interface while separating file, object, encoding, and verification sizes; and it provides an evidence-driven evaluation protocol covering provisioning, metadata, physical I/O, and agent execution. Our source audit also identifies integration gaps that must be closed before claiming an evaluated end-to-end system. We do not report an acceleration factor or claim reduced write amplification without measurements.

2 Background and Motivation

2.1 Workspaces as shared state and private changes

A workspace contains more than a version-controlled source tree. Tools may also inspect dependency directories, generated artifacts, build trees, and temporary files. The intended deployment chooses which directory subtrees are stable enough to publish as immutable baselines; other state remains mutable. Multiple tasks can reference the same published subtree without requiring their subsequent writes to become mutually visible.

The useful workload properties must be established empirically. We will measure the overlap of files and metadata read across tasks, the fraction of the initial namespace that each task modifies, and the lifetime of a shared baseline. These quantities distinguish a favorable workload from one in which tasks rewrite most files or repeatedly invalidate their dependencies. Agent framework papers establish that repository navigation and editing occur; they do not establish the frequency of BrewFS's storage bottlenecks. We therefore treat high sharing and sparse mutation as hypotheses rather than facts about all coding agents.

An earlier project pilot used DSH 0.1.5-rc.1 with the configured model `deepseek-v4-flash` over a WSL passthrough FUSE file system. It is a workload lead, not a BrewFS/RustFS performance measurement. This draft does not reuse its numerical results because the raw event records and timing definitions have not been revalidated here. The formal study will recover or recapture the traces and distinguish handler service time, overlapping I/O, tool time, and task wall-clock time.

2.2 Metadata sharing does not determine metadata access

Namespace organization and caching have long been central to file-system performance. TableFS organizes local metadata in a table; IndexFS combines partitioning, caching, and bulk insertion; LocoFS and InfiniFS revisit the organization of distributed metadata services [11, 13, 17, 18]. These systems caution against assuming that every metadata operation always requires a distinct network round trip. A fair comparison must include their relevant techniques and an equal cache budget.

BrewFS studies a restricted opportunity: selected lower subtrees cannot change after publication. Their lookup records need not participate in the same update protocol as the private upper. An immutable representation can carry directory entries, frequently used attributes, and data locators together, allowing a client to fetch enough information for several related operations. The benefit depends on access locality and cache state. For random point lookups, a group may fetch more bytes than an individual record. For a cold deep path, several index pages may be required. Both cases belong in the motivation study and the evaluation.

The property being tested is *lower-query independence*, not the elimination of a metadata service. Upper lookups, workspace control, reader pins, publication, and reclamation may still require mutable records. This distinction is also essential when comparing BrewFS with DeltaFS, whose private logs and selectively assembled namespaces already challenge a single global metadata ground truth [26].

2.3 Logical request size differs from physical work

There are at least five relevant sizes: an application's request, a file-system logical chunk, the finalized uploaded object, an erasure-coding block, and a per-shard verification/read frame. Compression, framing, and partial tail objects make the uploaded length differ from file length. Erasure coding then distributes each coding block across data and parity shards.

A range request can require a larger verified interval on each shard that serves it.

Storage systems already make different decisions about striping, coding, and redundancy transitions [1,5,8]. BrewFS's proposed connection is specifically between finalized object length and the backend's coding and verification geometry. It leaves the data/parity ratio fixed during a causal comparison. Smaller blocks do not by themselves reduce that ratio, and smaller verification frames add checksums and processing overhead.

Moreover, the client may fetch a full object even for a short application read. In that case, a smaller backend quantum may not reduce the bytes requested. We will trace the complete path, including the lower reader, upper integrity check, client cache, HTTP request, shard reads, and repair. This is necessary to determine whether a physical-layout optimization benefits the intended workspace workload.

3 Overview and System Contract

Let S be a pinned immutable snapshot and Δ_i the private upper of workspace i. Its logical view is

$$W_i = \mathrm{Resolve}(S, \Delta_i). \tag{1}$$

The design selects a read-only subtree for publication; the inspected workspace binds one precise lower manifest. This does not require a monolithic snapshot of the entire source volume, nor does it establish support for independently binding several manifests in one workspace. The upper records private names, current attributes, deletion state, and file-data mappings. New payload bytes are stored separately from those metadata records. Figure 1 shows the intended connection between the representation and the object layout.

The intended isolation property is view-local mutation: operations in W_i do not change S or another workspace's private state. This is a namespace and data-visibility property, not a complete security sandbox. Process isolation, network access, and escape prevention belong to the agent runtime. We do not infer their guarantees from the file-system overlay. Nor does the design require automatic merging of independent agents' edits or a transaction spanning several workspaces.

The immutable snapshot has an identity and authenticated roots. Clients bind to that identity instead of following a mutable "latest" pointer during each lookup. Upper records identify the workspace and its objects. The reader resolves upper state first and follows lower references only where the upper representation permits inheritance. Publication and reclamation must preserve the lifetime of every referenced immutable object.

For example, two workspaces can read the same dependency entry from a shared packed directory page. If one workspace overwrites an interval of that file, its private mapping records the new bytes while unchanged intervals still reference the lower. A later lower read follows its data locator to a packed frame, whose object is served using its persisted encoding block B and verification quantum q. This connects namespace sharing to the bytes read by storage. The connection is path-dependent: the inspected upper reader verifies a whole object before slicing it, which can prevent a short application read from becoming a short backend range.

Our audit distinguishes two writable workspace implementations. The older workspace path uses the ordinary VFS stack, while the experimental Entity path has a dedicated FUSE adapter and explicit current-state data mappings. Sections 4–5 explain the Entity model and packed snapshot contract. Section 7 states which connections are present, gated, or incomplete. This separation avoids describing properties of one path as guarantees of another.

The target correctness contract has three boundaries. Within a workspace, each supported operation must preserve namespace and data invariants under the implemented concurrency protocol. Across workspaces, changes remain private. Across failure and publication events, only a validated snapshot and durable referenced state may become visible. The latter requires explicit fault testing; the current fresh-only Entity mount does not yet establish crash takeover or reopen/resume guarantees.

4 Delta-Based Workspaces

4.1 Private current state over a pinned lower

The Entity representation stores the current state of changed entities in a private upper. A name record is either present, explicitly removed, or absent from the upper. A present record points to the workspace's entity; a deletion marker, or whiteout, suppresses the inherited name. Only absence permits fallback to the lower, subject to the directory's merge and opacity rules. Thus a deleted lower file cannot reappear merely because its upper payload has been reclaimed.

For a parent directory d and name n, the basic resolution rule is

$$R_i(d,n) = \begin{cases} e, & \Delta_i(d,n) = \mathrm{Present}(e), \\ \bot, & \text{the name is hidden in } W_i, \\ R_S(d,n), & \text{otherwise.} \end{cases} \tag{2}$$

The hidden case includes a whiteout and the applicable directory-opacity rules. The equation describes visibility, rather than prescribing how many database or object requests a lookup performs. The actual reader can reuse cached upper state and packed lower pages.

Directory enumeration combines upper and lower names under the same rule. It must suppress duplicates, apply deletions, and return attributes associated with the visible entity. An upper directory's current attributes are not an arithmetic merge of the two layers' attributes. Stable identity also matters: within a workspace, hard-link aliases must continue to

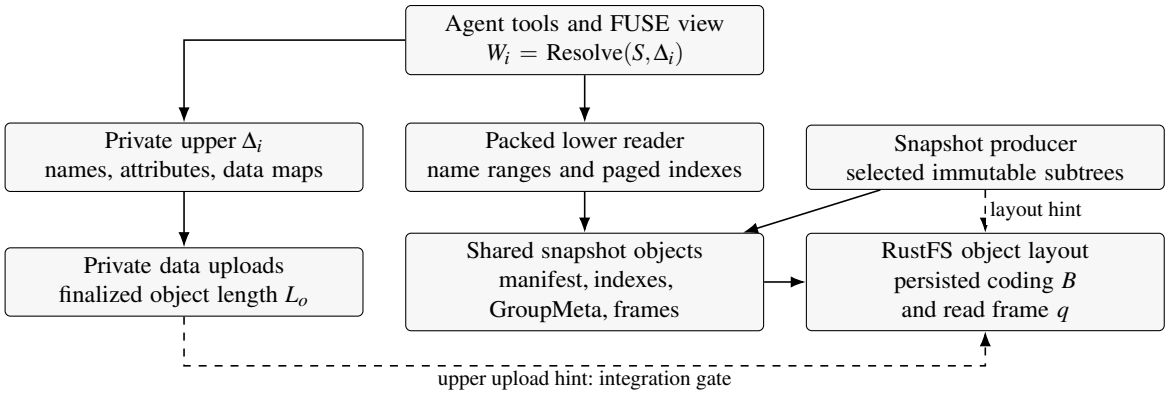


Figure 1: Design overview. Solid arrows denote the logical paths present in the inspected components, not completed runtime validation. Dashed arrows mark layout-hint connections that the selected Entity mount and packed producer must explicitly wire and verify. Private upper/control state still uses a metadata service. The shared snapshot includes metadata and file-data locators, rather than a single metadata packet.

refer to the same logical inode when private state is materialized. The selected runtime's supported operation set and directory limits are part of the experimental contract, rather than an assumption of complete POSIX compatibility.

4.2 Representing changed data without cloning the baseline

Private file contents use explicit mappings. For a changed logical chunk, the normalized map covers its applicable range with spans of three kinds: *Data*, which references newly stored upper bytes; *LowerRange*, which references inherited bytes in the pinned snapshot; and *Hole*, which reads as zeros. A missing interval in an explicit map must not be interpreted as an implicit lower reference. The distinction prevents stale baseline data from leaking into a region that has been overwritten or truncated.

Consider a file inherited from S. A write to a short interval materializes the required private attributes and mapping, stores the new bytes, and replaces the corresponding interval. Unchanged intervals can continue to reference S. This does not require a byte-difference algorithm or a copy of the entire lower file. It also does not imply that a 4 KiB application write always produces exactly a 4 KiB object: upload boundaries, format overhead, and the chosen data path still determine object size.

Truncation requires a separate inheritance boundary. The Entity model keeps a lower-visible size that can decrease but does not increase. After a file is truncated and later extended, the newly visible region reads as zeros or new upper data; it must not resurrect bytes that remain in the immutable lower. The ordinary file length alone cannot encode this history-sensitive rule.

These mappings create a useful separation between view changes and physical objects. Two workspaces may retain the same lower reference while storing different upper spans. A reader combines the spans needed for its request. An upper object and a lower packed frame can nevertheless have different integrity and fetch policies. In the inspected Entity path, upper-object integrity is checked over the full object, whereas the packed lower exposes authenticated frame reads. We preserve this distinction in Section 6 and the experiments.

4.3 Mutation ordering and lifecycle boundaries

The private metadata protocol must keep related namespace changes consistent. For example, rename can affect the old parent, new parent, and a replaced target; publishing a new data mapping must not expose incomplete payload. The Entity data path performs a durable conditional object upload before publishing the corresponding mapping. Its lease and authority checks are intended to fence stale writers. These code-level mechanisms still require concurrent-history and failure testing before a complete runtime guarantee can be claimed.

Snapshot ownership is equally important. An immutable object cannot be removed while a live workspace or reader still references it. A publication or ownership transition must retain old readers until they release their references, and cleanup must distinguish unreachable construction artifacts from live lower objects. Library support for ownership and recovery is not evidence that the mount lifecycle has implemented every transition. The current Entity mount is experimental and fresh-only: reopen, resume, and process-crash takeover remain integration obligations.

For live logical metadata bytes, the accounting identity is

$$M_{\text{live-meta}} = M_S + \sum_i M_{\Delta_i} + M_{\text{control}}. \tag{3}$$

Here M_S counts the shared lower's live metadata once, M_{Δ_i} counts workspace i's private names, attributes, deletion

records, and data mappings, and M_{control} counts bindings and ownership records not already included in either term. These are disjoint logical categories. Upper payload, encoding and checksum overhead, construction or unreclaimed objects, and caches are measured separately in the physical-space and resource accounting. This expression exposes the sharing benefit to test, not a measured saving. M_{Δ_i} contains current records and mapping overhead, not merely changed fields; it can grow as more entities and deletion records become private. We will compare the resulting costs with full materialization, reflinks, OverlayFS, and BranchFS under their respective visibility and durability contracts.

5 Packed Metadata

5.1 An immutable snapshot with access-oriented structure

Packed Metadata covers the metadata and access structure of selected read-only subtrees. It is not a small standalone packet containing a few attributes. The inspected format includes a manifest with authenticated roots, paged indexes, directory-oriented metadata pages, object containers, and file-data descriptors. Together these structures describe a complete immutable lower view and how its contents can be read.

A logical read group is organized around a parent directory and a range of raw entry names. Name order is a format-defined byte order, not a locale's collation. Group metadata brings together directory entries, frequently needed attributes, and locators for file data. A physical container can hold more than one logical group, so logical grouping and physical object size remain distinct decisions. Small contents may be inline, but metadata co-location, inlining, compression, and data co-packing are separate factors in our evaluation.

The current representation uses a PM11 manifest and IP06 hierarchical indexes. Its GroupMeta raw page is bounded at 256 KiB. Other indexes support different access paths, including inode lookup and data-frame location. Bounding a page does not bound the entire snapshot or force all directory entries into memory at mount time. Conversely, it does not make every operation a constant number of GETs: a cold lookup can traverse several pages, and a data read can require descriptor and payload access.

5.2 Lookup, attributes, enumeration, and read

For a name lookup, the lower reader identifies the directory's relevant range and traverses the authenticated index to its metadata page. The returned record can supply the common attributes and location information needed by subsequent operations. A direct getattr can instead use an inode index carrying hot attributes; it need not fetch a whole directory group. Figure 2 illustrates a typical name-based access, not all possible paths.

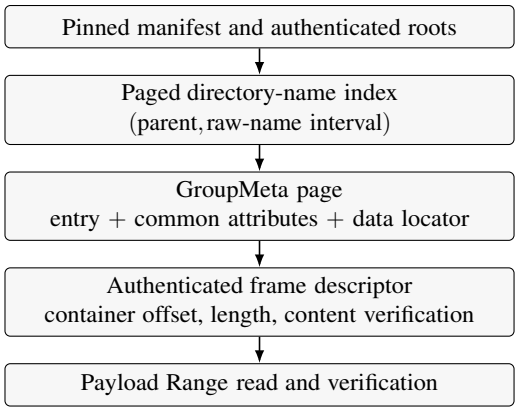


Figure 2: A representative lower access. A directory-name lookup descends indexed ranges and obtains an entry, common attributes, and a data locator. File reads use the authenticated frame descriptor and payload range. Direct inode-based getattr and other access paths may use separate indexes; this diagram is not a fixed GET-count claim.

Enumeration uses ordered ranges and page navigation to avoid requiring a single unbounded directory object. At the snapshot-reader layer, weighted indexes also support directory-ordinal access. These capabilities must be distinguished from the Entity bridge's current limits: the bridge has a bounded whole-directory interface and cannot yet expose arbitrary-sized directory enumeration. Large-directory snapshot microbenchmarks will therefore be reported separately from complete workspace measurements.

A file-data locator identifies an inline value or a referenced data region. For an external frame, the reader resolves and validates its descriptor, requests the applicable object range, and verifies the returned contents according to the snapshot format. Failed authentication or an unavailable referenced object is an error; it must not silently become a successful negative name lookup. An index's absence result and a storage failure are different outcomes.

For these supported lower queries, the needed namespace, attributes, and data locations reside in the snapshot rather than in per-file records in the mutable metadata database. The writable overlay may still check its private state first. We will instrument these two kinds of database traffic separately and test cold lower access while denying the lower-record path. Counting only cache hits would not establish this property.

5.3 Publication, ownership, and construction cost

Snapshot production begins from a fixed source view. The producer groups entries, builds metadata and data objects, and constructs the indexes and manifest. A safe publication contract requires all referenced dependencies to be durable and validated before the new binding becomes visible. Old readers

must retain their previous immutable view until it is safe to reclaim. A source that changes during capture requires an explicit consistency mechanism; an immutable output format alone does not supply it.

The repository contains producer, source-import, binding, journal, reader-pin, and collection components. The binding proof explicitly does not, by itself, prove source-view consistency or the closure of all uploaded dependencies. We therefore treat end-to-end publication and failure recovery as obligations for the selected integration, not as consequences of a successful manifest serialization. Ordinary close or fsync does not automatically rebuild a packed snapshot.

Publication can be expensive. Reading source metadata, sorting names, compressing data, uploading objects, and creating authenticated indexes all consume resources. The relevant comparison includes this cost and any rebuild or reclamation work over the snapshot's actual reuse lifetime. For each additive resource metric c, we will measure

$$C_c(K) = C_{c,\text{build}} + C_{c,\text{publish}} + \sum_{i=1}^{K} C_{c,i} + C_{c,\text{reclaim}}. \quad (4)$$

CPU-seconds, transferred bytes, and monetary cost are separate metrics; concurrent task wall-clock times are not summed into a batch makespan.

5.4 What the layout must earn

Immutable metadata on object storage is already a strong existing design space [2, 14, 26]. The specific case for BrewFS rests on the cost of its directory-oriented joint records and paged access under the writable-upper contract. We will compare against an immutable sorted-KV lower with identical record content and cache capacity, and against at least one demand-loaded immutable-image system where its operation contract is compatible. The sorted-KV reference is a mechanism control, not a claimed reproduction of DeltaFS.

The layout can lose when accesses have little locality, object latency dominates a cold index walk, or snapshots are rebuilt too frequently. These outcomes will define its operating region. They should not be hidden by reporting only warm directory scans.

6 Variable-Granularity Object Storage

6.1 Choosing at the finalized-object boundary

Let L_o be an object's finalized upload length, after the applicable compression, headers, and framing. The client computes a hint $h(L_o)$ when this length is known. It does not substitute the original file length, the size of a FUSE request, or reserved local staging space. One file may produce several objects, and a tail object can require a different decision from the preceding full object.

The S3 adapter supports default, automatic, and fixed hint modes. Its fixed choices are 64 KiB, 256 KiB, 1 MiB, and 4 MiB. The current automatic policy is deliberately simple:

$$h(L_o) = \begin{cases} \text{omit hint}, & L_o \le 1\,\text{MiB}, \\ 1\,\text{MiB}, & L_o > 1\,\text{MiB}. \end{cases} \quad (5)$$

This is not an adaptive optimizer. In particular, the audited backend's default coding block is also 1 MiB, so automatic mode may leave the effective coding block unchanged. A fixed-size sensitivity sweep demonstrates layout capability; it does not by itself demonstrate that this automatic policy outperforms the default.

The adapter applies the policy to ordinary PUT, vectored PUT, non-strict create-only PUT, and multipart initiation using the complete object's known length. The strict durable create-only path is an audited exception and an integration gate (Section 7). It must not choose a new object layout from each multipart part. Retries must preserve the chosen policy for that object. RustFS validates the hint under its admission and compatibility rules, then persists the effective layout. A client-sent header is therefore insufficient evidence that a particular layout was used.

6.2 Encoding block and physical verification quantum

Let B be the effective erasure-coding block size, d the number of data shards, and p the number of parity shards. For a compatible full stripe, let s denote the logical size of one shard. The physical verification/read quantum q is chosen within this encoding geometry. The inspected selector requires

$$q \mid s, \qquad qd \mid B. \quad (6)$$

Thus q is a separate decision with constraints, rather than an arbitrary independent byte count. It should also not be confused with a packed snapshot's compression or authentication frame.

The backend contains gated code for selecting and persisting a smaller q for eligible new objects. Both enablement and fleet-confirmation gates are off by default. Inline objects and protected/shard-integrity write modes follow their applicable existing paths. If the selected q equals the logical shard size, no new framing marker is needed. Table 1 shows examples from the selector for full stripes in a 2+2 geometry; these are code-derived configurations, not benchmark results.

Reads and healing interpret the layout stored with the object, rather than reinterpreting old data using the current client's hint or current write gates. This is necessary for mixed old and new objects, restarts, and gradual rollout. The evaluation will explicitly exercise these cases, including a failed or corrupted shard and a later repair.

B	$s = B/2$	selected q	Smaller frame?
64 KiB	32 KiB	16 KiB	Yes
256 KiB	128 KiB	64 KiB	Yes
1 MiB	512 KiB	512 KiB	No
4 MiB	2 MiB	1 MiB	Yes

Table 1: Eligible full-stripe examples from the audited backend selector, with $d = 2$, $p = 2$, and both new-write gates enabled. Object eligibility and persisted metadata still require validation. Other shard geometries can produce different outcomes.

6.3 From a logical range to physical work

For a nonempty interval of length $\ell > 0$ beginning at offset a within one shard, the ideal number of frames covered is

$$n_q(a,\ell) = \left\lceil \frac{a+\ell}{q} \right\rceil - \left\lfloor \frac{a}{q} \right\rfloor. \tag{7}$$

Ignoring tail and batching details, the covered data is approximately $n_q q$, plus per-frame checksum bytes. Total physical work sums over the data and, when required, parity shards actually read. The application request length cannot simply be substituted for ℓ: coding, compression, and the client's requested range determine the shard interval.

This model explains a potential reduction in over-read, not a performance result. Smaller frames can require more checksum operations and metadata; small coding blocks can increase per-block overhead. With fixed (d, p), the nominal redundancy ratio remains $(d+p)/d$. The backend also handles tail stripes using their actual remaining length, so a large B does not justify assuming that every tail object wastes an entire padded stripe. Read, upload, persistent-write, space, and repair amplification must be measured separately.

6.4 Client fetch policy is a separate intervention

In the ordinary VFS/ObjectBlockStore path, enabling a hint also selects whole-object fetch preferences and associated SingleFlight/LRU behavior. Even an object for which automatic mode omits an upload header can be read under that mount-level policy. A comparison of "hint off" and "hint on" can therefore change caching and duplicate GETs without changing B. Those effects cannot be attributed to erasure-code granularity.

The experimental Entity upper has a different limitation: its reader obtains the full referenced object, checks its SHA-256, and then extracts the requested part. Its writes also bypass the ordinary VFS's buffered 4 MiB aggregation. The mounted Entity path accepts writes of at most 1 MiB and directly uploads the resulting segments. Its upper objects therefore do not reach the current automatic policy's above-1-MiB branch; inline handling may apply as well. A larger logical file does not remove this limit. The packed lower's authenticated frame requests are a more direct initial candidate for testing physical range granularity.

We will first verify (B, q) effects with a fixed S3 request stream, then repeat the relevant cases through the lower and upper file-system paths. Making upper reads independently verifiable at smaller granularity would be an additional design change. We will not obtain a favorable range-read result by silently removing the current integrity check.

7 Implementation and Integration Status

The audit fixes the file-system and backend revisions rather than assuming the current default branches have the same behavior. The inspected revisions are BrewFS `518e7cc39b2f` and the RustFS research fork `cc4926fac110`.[1] The accompanying evidence ledger distinguishes specification, code presence, runtime connection, and measured behavior. This preparation did not execute new mounts, fault tests, or performance runs.

Workspace paths. The ordinary volume and legacy workspace paths use the VFS infrastructure. The Entity workspace instead uses a dedicated FUSE adapter, direct I/O, zero metadata TTL, and no ordinary VFS slice-cache/writeback path. It is explicitly experimental and fresh-only. Its current API bounds include a 16 MiB I/O/chunk limit, a 1 MiB FUSE maximum write, a 4,096-entry complete directory interface, and a packed-owner census bounded at 256 active owners per store. These limits constrain the first evaluation; exceeding them is a separate implementation task.

Snapshot components. The packed reader, producer, authenticated indexes, and supporting publication components are present. A standalone read-only mount is available, but its current CLI rejects nonzero payload-cache and prefetch budgets. A cache sweep must therefore distinguish supported metadata caches from a new payload-cache implementation. Native Base supplies another data path and is not interchangeable with the Entity workspace in an ablation.

Layout integration gates. Three inspected connections prevent us from claiming a complete combined runtime. The Entity mount's S3 configuration does not propagate the ordinary CLI's EC hint; the packed fixture producer also uses a default hint setting; and the strict durable create-only upload used by EntityData does not attach the hint metadata. The last path

[1] Fixed sources: `https://github.com/brewfs/brewfs/tree/518e7cc39b2ff0b421ecf0f656de6d468cacb9b8` and `https://github.com/Ivanbeethoven/rustfs/tree/cc4926fac1107aee76540f28bf22dbd47fcbca24`.

must retain its conditional-create and precondition-failure semantics when instrumented or extended.

Even after these connections are supplied, the Entity FUSE maximum write and direct upload formation produce upper objects no larger than 1 MiB in that path. The current automatic rule only emits a hint above 1 MiB. Consequently, a large logical file is not sufficient to exercise automatic upper-object B selection. Initial policy experiments must use packed containers or the ordinary finalized upload path; Entity upper experiments can test supported fixed hints, actual small-object behavior, or an explicitly new aggregation design. We will not imply that this extra design already exists.

Backend gates and measurement. RustFS contains B parsing/persistence and q-aware write, read, and healing code. New q writes are disabled by default and were disabled in the historical configuration inspected by this audit. A valid experiment will record actual object (B, q), eligibility, and physical read counters rather than only flags. Ordinary-VFS write-budget release and shared-buffer replies are additional retained optimizations, not consequences of changing B. They must remain constant when isolating layout effects.

The AX directory provides deployment and mount contracts; its smoke path can use a durable directory without a BrewFS mount. Formal end-to-end experiments will verify mount identity and correlate agent tool operations with the file-system and object counters. Starting an AX task alone is insufficient integration evidence.

8 Evaluation Plan

This section is a protocol for future experiments. It contains no generated measurements or assumed speedups. The companion experiment matrix defines E00–E16, including instruments, negative cases, and execution gates. Table 2 connects the principal questions to those experiments.

8.1 Environment and baseline contracts

Our proposed primary testbed has two client machines, one dedicated metadata machine, and four RustFS storage machines connected by 25 GbE. A practical target is 24–32 physical cores and 128 GiB memory per client, with independent NVMe storage and fixed cache allocations. The initial storage geometry is one drive per storage host in a verified 2+2 set. These are requested resources, not a report of machines already used. A single-host four-drive setup serves only as a functional and mechanism testbed; it cannot establish independence of machine failure domains.

We will record processor, memory, disks, network, OS/kernel, FUSE settings, toolchain, complete configuration, build features, immutable image digests, and source revisions. Durability, compression, cache budget, readahead, direct I/O, metadata TTL, inline thresholds, and backend gates are part of the treatment definition. Larger coding geometries will be separate experiments because they can change whether q selection actually reduces a frame.

External baselines include full copying, reflinks where supported, OverlayFS, BranchFS, and an object-backed file system such as JuiceFS [7] sharing the same backend resources. At least one mature immutable lower, such as Nydus or CernVM-FS, will be tested with a compatible writable layer. An immutable sorted-KV lower with equal records and cache capacity provides a specific layout control. A real DeltaFS comparison depends on its operation and workflow compatibility; we will not label our sorted-KV control "DeltaFS."

We will compare local workspace service separately from remote durable service. An OverlayFS local fsync is not equivalent to remote coded-object completion. Similarly, BranchFS's ephemeral branch uses different fsync and commit semantics [20]. Where a baseline needs a checkpoint to meet the selected persistence boundary, its completion cost is included. If equivalent semantics cannot be provided, we report separate operating points instead of one unconditional speed ranking.

8.2 Workloads and measurements

RQ1. The workload study will collect repository navigation, reading, editing, build/test, and cleanup activity from a pinned agent harness and model configuration. It will measure read-set overlap, directory locality, reuse distance, mutation footprint, and per-task operation distributions. Expected lookup and attribute probes are distinguished from I/O failures. An instrumentation-on/off control will quantify trace overhead and loss.

The proposed expanded panel contains 20 compatible tasks each from SWE-bench Verified, a fixed public SWE-bench Pro version [19], and a multilingual set informed by Multi-SWE-bench [6, 25]. Exact task lists and revisions will be frozen after capability screening. An official audit has identified limitations in using SWE-bench Verified to measure frontier model ability [15]; we use it as a reproducible workload source and retain other task families. Filesystem performance does not establish improved reasoning ability.

The minimum live-agent core will use 20 prespecified tasks, three systems, and three independent repetitions at one concurrency setting: 180 runs. The 60-task extension requires 540 runs for that same setting, before any additional model or concurrency sweep. Pilot task duration and token use will determine a frozen budget. Broad concurrency sweeps first use replay; they are not silently multiplied across every live condition.

RQ2. Provisioning experiments vary lower size, workspace count, and modification fraction. Initial values are 10^4–10^6 files, 1–128 workspaces, and 0–50% modified files. We measure control-plane return, mount readiness, first navigation,

RQ	Question	Primary evidence	Experiments
1	Is the workload favorable to shared immutable state?	Cross-task overlap, mutation footprint, stage-specific metadata and I/O distributions	E01–E02
2	When does the delta representation reduce workspace cost?	Time to usable workspace, private metadata, modification and lifecycle costs	E03–E04
3	What does the packed lower save, and at what cost?	Lower-KV calls, object requests/bytes, latency, cache effects, construction amortization	E05–E07
4	What changes when (B,q) and client policy change?	Persisted geometry, physical bytes, verification/EC CPU, durable writes and repair	E08–E10
5	Do the mechanisms provide additional combined value?	Legal factorial or nested comparisons under one runtime contract	E11
6	Do the effects matter for complete multi-agent tasks?	Deterministic replay, live-task makespan, resource use, output-quality controls	E12–E13
7	Which semantics and failure boundaries hold?	Differential operations, fault injection, integrity, reclamation and long-run resource bounds	E14–E16

Table 2: Research questions and planned evidence. E00 first checks that each configuration is connected and produces the claimed effective treatment.

and first durable edit separately. Prepositioned and cold-staged baselines are separate cases. Churn experiments add deletion, rename, truncation, and repeated lifecycles, subject to the selected implementation's ownership and cleanup support.

RQ3. Lower lookup experiments compare mutable per-file KV, immutable sorted-KV, and the packed layout using equivalent values and cache budgets. They include positive and negative lookup, direct getattr, enumeration, lookup-plus-read, deep paths, and wide directories. The snapshot reader's large-directory study is separated from the Entity bridge's 4,096-entry limit. We count upper/control/lower KV requests independently, as well as index, GroupMeta, descriptor, and payload object requests and bytes.

Cold-client, index-warm, metadata-warm, and data-warm states are explicitly defined; backend page-cache state is another variable. Unsupported payload-cache controls are not treated as available flags. Snapshot construction, publication, rebuilding, and reclamation enter each resource accounting metric. We report a break-even reuse count only if the measured net per-workspace benefit is positive.

RQ4. A backend-only study uses a fixed S3 request stream before adding the file-system path. Object sizes cross inline, 1 MiB, coding-block, and multipart boundaries. B settings include default, current automatic, and the four supported fixed values. q settings are legacy or the implemented selector, grouped by *persisted* geometry. If automatic and default produce the same (B,q), they are a no-treatment control, not evidence for an adaptive layout benefit.

Reads vary length, alignment, coding/frame-boundary crossing, and healthy versus degraded state. The primary measurements are physical data/parity read bytes, checksum work, EC CPU, HTTP payload, request counts, and latency. Client Range and full-object policies are then separated while keeping cache admission and coalescing constant or exposing them as additional factors. Upper and lower results are separate because their integrity and upload paths differ.

Writes include append, overwrite, tails, multipart retry, and frequent durability requests. We record accepted writes, staging, remote completion, metadata commit, and drained background debt. Repair experiments measure read/write/network work and interference with healthy requests. The paper will claim lower write amplification only if these measurements support it; a read benefit can coexist with increased checksum or write cost.

8.3 Composition and complete agent execution

RQ5. Mechanism comparisons must retain one runtime contract. Switching from a buffered ordinary volume to a direct-I/O Entity mount is not a pure delta ablation. We will implement or validate matched controls for complete metadata materialization versus shared lower state, lower layout, and backend geometry. Only configurations that can actually be constructed enter a factorial comparison. If a packed lower structurally requires the shared-workspace path, its effect is conditional and the design is nested. Where full crossing is possible, task-level log-latency interactions will test whether combined effects differ from independent component effects.

RQ6. Deterministic replay and live agents answer different questions. Replay freezes tool inputs, write contents, and dependencies, preserving file-tree and output checks. A FUSE log containing only inode IDs and request lengths is not necessarily replayable; paths, handle lifetimes, payload references, initial state, and ordering must be recovered or recaptured. Arrival-driven and closed-loop replay are reported separately.

Live AX/DSH experiments keep the task, model identifier, prompt, tools, token/time budgets, and concurrency policy

fixed. Mount probes must verify that workspaces and subprocesses reach the intended file system rather than a fallback directory. We measure batch makespan, tasks per hour, provisioning, resource use, and the traced storage critical path. Model waiting and I/O can overlap, so their durations cannot simply be added. Output correctness is checked with the task harness; an exit code of zero is not a resolved issue.

8.4 Correctness, failures, and analysis

RQ7. Differential and known-answer tests cover whiteouts, same-name recreation, lower-file modifications, truncate/extend, holes, links, rename-overwrite, open-unlink-read, and cross-workspace visibility within the supported contract. Fault injection targets object upload, manifest and binding publication, mapping commit, process restart, missing objects, frame corruption, and repair interruption. Fresh-only Entity behavior is reported as such; reopen or crash-takeover experiments require a completed runtime extension. Fail-closed behavior is distinct from successful recovery.

Long runs cover multiple cache and cleanup cycles and track tail latency, memory, open descriptors, owners, pending uploads, retained objects, and reclamation backlog. Background publishing or repair is added only where the selected runtime supports it. High mutation, poor locality, frequent snapshot rebuilding, and large sequential reads are explicit boundary workloads, not exclusions from the evaluation.

The independent unit is a run or a task, not each of its millions of requests. Initial plans use at least ten independent microbenchmark blocks and three live repeats per selected task; the formal sample sizes are frozen after a variance pilot. Conditions are interleaved in randomized host/time blocks. We report absolute effects, paired ratios, and 95% task- or repository-clustered bootstrap intervals. Repeated task outcomes are not treated as independent binary pairs.

For live speed comparisons, the primary endpoint includes all assigned tasks under the same deadline; failed and timed-out attempts remain in the throughput and completion accounting. Completed-only latency is secondary and accompanied by success rates and failure causes. Early failures do not count as fast completions. This protocol prevents storage behavior or model failures from creating an apparent speedup through selective omission.

9 Related Work and Limits of the Claim

Workspaces and agent-native file systems. OverlayFS supplies the layered semantics that motivate the private upper and shared lower [12]. BranchFS provides copy-on-write branch contexts for agentic exploration, including a commit/abort lifecycle; its workshop prototype and proposed kernel abstraction must be distinguished [20]. YoloFS studies staging, snapshots, and permissions to improve visibility and control over agent effects [27]. AgentFS also documents a file-system and tool-state abstraction for agents [3]. These efforts make an "agent filesystem" label insufficient as a novelty claim. BrewFS instead targets the costs of shared remote lower metadata and object access. It does not replace process isolation or provide all of these systems' user-control and commit semantics.

Namespace and metadata organization. TableFS, IndexFS, LocoFS, and InfiniFS demonstrate that record organization, partitioning, and caching can substantially change metadata access paths [11, 13, 17, 18]. Recent systems also revisit hierarchical object metadata and distributed update coordination [10, 23]. We compare mechanisms at their actual service boundaries instead of assuming that every baseline performs a remote per-file lookup.

DeltaFS is a particularly close conceptual predecessor. It uses private namespaces and immutable mutation logs over shared storage, with selective composition of prior state [26]. BrewFS cannot claim private deltas or object-resident snapshots as inventions. Its proposed distinction is a directory-oriented lower layout that combines common attributes and data locators in paged access units, composed with an online private upper and a backend layout interface. Whether this particular combination is worth its complexity is an evaluation question.

Immutable distribution and storage backends. CernVM-FS provides demand-driven software distribution with indexed catalogs, while Nydus describes metadata/data organization for lazy image access [2, 14]. Slacker and DADI also address image distribution and startup costs [4, 9]. These are stronger points of comparison for immutable baselines than a model that downloads every file before use. The experiment plan therefore requires a mature lazy-lower baseline and counts image construction and staging separately.

Distributed systems such as Ceph and Tectonic separate aspects of metadata and data services [16, 22]; this separation itself is not a contribution of BrewFS. Erasure-coded storage research considers repair cost, changing redundancy over a file's lifetime, and the relation between data placement and redundancy [1, 5, 8]. Okapi's separation of striping and redundancy grouping is different from our (B, q) distinction, although both motivate making coupled decisions explicit. We keep code rate fixed and measure checksum, tail, and repair cost rather than inferring a lower redundancy ratio from smaller blocks.

Scope and unresolved evidence. The design is most plausible for stable shared baselines and sparse private changes. A workload with poor name locality, high mutation, or rapid snapshot churn may favor existing mutable metadata services or local overlays. The current Entity mount limits scale and lifecycle guarantees; the current automatic B policy can be

identical to the default, and its upper objects do not exercise the above-1-MiB branch through ordinary FUSE writes. These are material limits, not editorial details. Results must determine whether all three title mechanisms deserve equally strong performance claims.

10 Conclusion

BrewFS investigates how private workspace changes, shared immutable metadata, and finalized object layouts can be organized under one file-system design. Delta-based views preserve independent changes, packed snapshots provide directory-oriented lower access, and the backend interface makes encoding and physical verification geometry explicit. The source audit establishes concrete components and identifies the connections and lifecycle support that still require work. The planned evaluation will determine when the design reduces total workspace and storage cost, how much of that effect survives complete agent execution, and where existing approaches remain preferable. No quantitative performance conclusion is drawn in this draft.

References

[1] Sanjith Athlur, Timothy Kim, Saurabh Kadekodi, Francisco Maturana, Xavier Ramos, Arif Merchant, K. V. Rashmi, and Gregory R. Ganger. Okapi: Decoupling data striping and redundancy grouping in cluster file systems. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 897–914. USENIX Association, 2025.

[2] Jakob Blomer, Carlos Aguado-Sánchez, Predrag Buncic, and Artem Harutyunyan. Distributing LHC application software and conditions databases using the CernVM file system. *Journal of Physics: Conference Series*, 331(4):042003, 2011.

[3] Pekka Enberg and Glauber Costa. The missing abstraction for AI agents: The agent filesystem. Turso engineering blog, November 2025. Published 2025-11-13; official software announcement, not a peer-reviewed paper.

[4] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Slacker: Fast distribution with lazy Docker containers. In *14th USENIX Conference on File and Storage Technologies (FAST 16)*, pages 181–195. USENIX Association, 2016.

[5] Cheng Huang, Huseyin Simitci, Yikang Xu, Aaron Ogus, Brad Calder, Parikshit Gopalan, Jin Li, and Sergey Yekhanin. Erasure coding in Windows Azure Storage. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 15–26. USENIX Association, 2012.

[6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[7] Juicedata, Inc. JuiceFS architecture. JuiceFS Community Edition documentation. Official technical documentation; page last updated 2025-11-06, accessed 2026-10-22.

[8] Timothy Kim, Sanjith Athlur, Saurabh Kadekodi, Francisco Maturana, Dax Delvira, Arif Merchant, Gregory R. Ganger, and K. V. Rashmi. Morph: Efficient file-lifetime redundancy management for cluster file systems. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP '24)*, pages 330–346. Association for Computing Machinery, 2024.

[9] Huiba Li, Yifan Yuan, Rui Du, Kai Ma, Lanzheng Liu, and Windsor Hsu. DADI: Block-level image service for agile and elastic application deployment. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 727–740. USENIX Association, 2020.

[10] Jiahao Li, Biao Cao, Jielong Jian, Cheng Li, Sen Han, Yiduo Wang, Yufei Wu, Kang Chen, Zhihui Yin, Qiushi Chen, Jiwei Xiong, Jie Zhao, Fengyuan Liu, Yan Xing, Liguo Duan, Miao Yu, Ran Zheng, Feng Wu, and Xianjun Meng. Mantle: Efficient hierarchical metadata management for cloud object storage services. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP '25)*, pages 928–943. Association for Computing Machinery, 2025.

[11] Siyang Li, Youyou Lu, Jiwu Shu, Yang Hu, and Tao Li. LocoFS: A loosely-coupled metadata service for distributed file systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17)*, 2017.

[12] Linux Kernel Contributors. Overlay filesystem. Linux kernel documentation. Official technical documentation, accessed 2026-10-22; pin the tested kernel and mount options in the evaluation.

[13] Wenhao Lv, Youyou Lu, Yiming Zhang, Peile Duan, and Jiwu Shu. InfiniFS: An efficient metadata service for large-scale distributed filesystems. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*, pages 313–328. USENIX Association, 2022.

[14] Nydus Authors. Registry accelerated file system. Nydus official documentation. Official technical documentation, accessed 2026-10-22; not a peer-reviewed paper.

[15] OpenAI. Why SWE-bench verified no longer measures frontier coding capabilities. OpenAI research announcement, February 2026. Published 2026-02-23; official benchmark audit, not a peer-reviewed paper.

[16] Satadru Pan, Theano Stavrinos, Yunqiao Zhang, Atul Sikaria, Pavel Zakharov, Abhinav Sharma, Shiva Shankar P, Mike Shuey, Richard Wareing, Monika Gangapuram, Guanglei Cao, Christian Preseau, Pratap Singh, Kestutis Patiejunas, JR Tipton, Ethan Katz-Bassett, and Wyatt Lloyd. Facebook's Tectonic filesystem: Efficiency from exascale. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 217–231. USENIX Association, 2021.

[17] Kai Ren and Garth Gibson. TABLEFS: Enhancing metadata efficiency in the local file system. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 145–156. USENIX Association, 2013.

[18] Kai Ren, Qing Zheng, Swapnil Patil, and Garth Gibson. IndexFS: Scaling file system metadata performance with stateless caching and bulk insertion. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '14)*, 2014.

[19] Scale AI. SWE-bench pro v2. Official dataset card on Hugging Face, September 2026. Version 2.0.0, released 2026-09-22; official dataset, accessed 2026-10-22.

[20] Cong Wang and Yusheng Zheng. Fork, explore, commit: OS primitives for agentic exploration. In *Agentic OS Workshop at ASPLOS 2026*, March 2026. Workshop paper; arXiv:2602.08199v2. The proposed branch() syscall is not implemented in the evaluated prototype.

[21] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[22] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*, pages 307–320. USENIX Association, 2006.

[23] Jingwei Xu, Mingkai Dong, Qiulin Tian, Ziyi Tian, Tong Xin, and Haibo Chen. SwitchFS: Asynchronous metadata updates for distributed filesystems with in-network coordination. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*. Association for Computing Machinery, 2026.

[24] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[25] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Shulin Xin, Linhao Zhang, Qi Liu, Aoyan Li, Lu Chen, Xiaojian Zhong, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Ming Ding, and Liang Xiang. Multi-SWE-bench: A multilingual benchmark for issue resolving. In *Advances in Neural Information Processing Systems*, volume 38, 2025. Datasets and Benchmarks Track.

[26] Qing Zheng, Charles D. Cranor, Gregory R. Ganger, Garth A. Gibson, George Amvrosiadis, Bradley W. Settlemyer, and Gary Grider. DeltaFS: A scalable no-ground-truth filesystem for massively-parallel computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, 2021.

[27] Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don't let AI agents YOLO your files: Information and control in agent-native filesystems. In *Proceedings of the 32nd ACM Symposium on Operating Systems Principles (SOSP '26)*. Association for Computing Machinery, 2026.